

5 Divide & Conquer

Efficient Algorithms · Wintersemester 2026–27

Prof. Dr. Sebastian Wild

CS566 (Wintersemester 2026–27)
Philipps-Universität Marburg
version **2026-11-09 16:17**

Learning Outcomes

Unit 5: ***Divide & Conquer***

1. Know the steps of the Divide & Conquer paradigm.
2. Be able to solve simple Divide & Conquer recurrences.
3. Be able to design and analyze new algorithms using the Divide & Conquer paradigm.
4. Know the performance characteristics of selection-by-rank algorithms.
5. Know the divide and conquer approaches for integer multiplication, matrix multiplication, finding majority elements, and the closest-pair-of-points problem.

Outline

5 Divide & Conquer

5.1 Divide & Conquer Recurrences

5.2 Order Statistics

5.3 Linear-Time Selection

5.4 Fast Multiplication

5.5 Majority

5.6 Closest Pair of Points in the Plane

Divide and conquer

Divide and conquer *idiom* (Latin: *divide et impera*)
to make a group of people disagree and fight with one another
so that they will not join together against one (Merriam-Webster Dictionary)

⇝ in politics & algorithms, many independent, small problems are better than one big one!

Divide and conquer

Divide and conquer *idiom* (Latin: *divide et impera*)
to make a group of people disagree and fight with one another
so that they will not join together against one (Merriam-Webster Dictionary)

⇝ in politics & algorithms, many independent, small problems are better than one big one!

Divide-and-conquer algorithms:

1. Break problem into smaller, independent subproblems. (Divide!)
2. Recursively solve all subproblems. (Conquer!)
3. Assemble solution for original problem from solutions for subproblems.

Divide and conquer

Divide and conquer *idiom* (Latin: *divide et impera*)
to make a group of people disagree and fight with one another
so that they will not join together against one (Merriam-Webster Dictionary)

⇝ in politics & algorithms, many independent, small problems are better than one big one!

Divide-and-conquer algorithms:

1. Break problem into smaller, independent subproblems. (Divide!)
2. Recursively solve all subproblems. (Conquer!)
3. Assemble solution for original problem from solutions for subproblems.

Examples:

- Mergesort
- Quicksort
- Binary search
- (arguably) Tower of Hanoi

Clicker Question



Have you seen the *Master Method* before?

- **A** Sure, could apply it blindfolded
- **B** Vaguely remember
- **C** Never heard of it



→ *sli.do/effalg*

5.1 Divide & Conquer Recurrences

Back-of-the-envelope analysis

- before working out the details of a D&C idea,
  it is often useful to get a quick indication of the resulting performance
  - don't want to waste time on something that's not competitive in the end anyways!
- since D&C is naturally recursive, running time often not obvious
  instead: given by a recursive equation

Back-of-the-envelope analysis

- before working out the details of a D&C idea,
  it is often useful to get a quick indication of the resulting performance
  - don't want to waste time on something that's not competitive in the end anyways!
- since D&C is naturally recursive, running time often not obvious
  instead: given by a recursive equation
- unfortunately, rigorous analysis often tricky
  - Remember mergesort?

$$C(n) = \begin{cases} 0 & n \leq 1 \\ C(\lfloor n/2 \rfloor) + C(\lceil n/2 \rceil) + 2n & n \geq 2 \end{cases}$$

$$\rightsquigarrow \quad C(n) = 2n \lg(n) + \left(2 - \{\lg(n)\} - 2^{1-\{\lg(n)\}}\right)2n \quad 😱$$

$$= \Theta(n \log n) \quad 😌$$

Back-of-the-envelope analysis

- before working out the details of a D&C idea,
  it is often useful to get a quick indication of the resulting performance
  - don't want to waste time on something that's not competitive in the end anyways!
- since D&C is naturally recursive, running time often not obvious
  instead: given by a recursive equation
- unfortunately, rigorous analysis often tricky
  - Remember mergesort?

$$C(n) = \begin{cases} 0 & n \leq 1 \\ C(\lfloor n/2 \rfloor) + C(\lceil n/2 \rceil) + 2n & n \geq 2 \end{cases}$$

$$\rightsquigarrow \quad C(n) = 2n \lg(n) + \left(2 - \{\lg(n)\} - 2^{1-\{\lg(n)\}}\right)2n \quad 😱$$
$$= \Theta(n \log n) \quad 😌$$

- the following method works for many typical cases to give the right **order of growth**

The Master Method

- Assume a stereotypical D&C algorithm:
  - recursive calls on a subproblems of size n/b each (for $n > n_0$)
  - non-recursive "conquer" cost $f(n) > 0$ for $n > n_0$ (any $f : \mathbb{R}_{>0} \to \mathbb{R}_{>0}$)
  - base cases: $T(n) = d_n \geq 0$ for $n \leq n_0$ (arbitrary constants)

Mergesort

$a = 2$

$b = 2$

$f(n) = 2n$ // element visit

$n_0 = 1$

$d_0 = d_1 = 0$

The Master Method

- Assume a stereotypical D&C algorithm:
  - recursive calls on a subproblems of size n/b each (for $n > n_0$)
  - non-recursive "conquer" cost $f(n) > 0$ for $n > n_0$ (any $f : \mathbb{R}_{>0} \to \mathbb{R}_{>0}$)
  - base cases: $T(n) = d_n \geq 0$ for $n \leq n_0$ (arbitrary constants)

$\rightsquigarrow$ running time $T(n)$ satisfies

$$T(n) = \begin{cases} d_n & n \leq n_0 \\ a \cdot T\left(\frac{n}{b}\right) + f(n) & n > n_0 \end{cases}$$

The Master Method

- Assume a stereotypical D&C algorithm:
  - recursive calls on a subproblems of size n/b each (for $n > n_0$)
  - non-recursive "conquer" cost $f(n) > 0$ for $n > n_0$ (any $f : \mathbb{R}_{>0} \to \mathbb{R}_{>0}$)
  - base cases: $T(n) = d_n \geq 0$ for $n \leq n_0$ (arbitrary constants)

$n = b^k$

$\rightsquigarrow$ running time $T(n)$ satisfies

$$T(n) = \begin{cases} d_n & n \leq n_0 \\ a \cdot T\left(\frac{n}{b}\right) + f(n) & n > n_0 \end{cases}$$

$f(n)$ // $T(n) = 2T(n/4) + \sqrt{n}$.

$c = \log_b(a) = \log_4(2) = \frac{1}{2}$

Case (b)

$T(n) = \Theta(\sqrt{n} \log n)$

Theorem 5.1 (Master Theorem)

With $c := \log_b(a)$, we have for the above recurrence:

(a) $T(n) = \Theta(n^c)$ if $f(n) = O(n^{c-\varepsilon})$ for constant $\varepsilon > 0$.

(b) $T(n) = \Theta(n^c \log n)$ if $f(n) = \Theta(n^c)$.

(c) $T(n) = \Theta(f(n))$ if $f(n) = \Omega(n^{c+\varepsilon})$ for constant $\varepsilon > 0$ **and** f satisfies the *regularity condition* $\exists n_1, \rho < 1 \; \forall n \geq n_1 : \; a \cdot f\left(\frac{n}{b}\right) \leq \rho f(n)$.

Clicker Question

Solve the recurrence $T(n) = 2T(n/4) + \sqrt{n}$.



A $\Theta(\sqrt{n})$

B $\Theta(\sqrt{n} \log n)$

C $\Theta(n)$

D $\Theta(n^2)$

E $\Theta(\log n)$

F No idea



→ *sli.do/effalg*

Clicker Question

Solve the recurrence $T(n) = 2T(n/4) + \sqrt{n}$.



- **A** ~~$\Theta(\sqrt{n})$~~
- **B** $\Theta(\sqrt{n} \log n)$ ✓
- **C** ~~$\Theta(n)$~~
- **D** ~~$\Theta(n^2)$~~
- **E** ~~$\Theta(\log n)$~~
- **F** ~~No idea~~



→ *sli.do/effalg*